CS 470 Final Reflection

Sabrina Warner

October 25, 2026

https://youtu.be/_VjLkfIyCTY

Experiences and Strengths

CS 470 has helped me better understand how full-stack applications can be developed, deployed, and maintained using cloud services. Before this course, I had experience with programming and web development through my Computer Science degree, but this course gave me more hands-on experience with cloud infrastructure and how different services work together. These skills will help me as I work toward starting a career in technology.

One of the most useful things I learned was how to work with Docker and containerization. I worked with Dockerfiles for the frontend and backend of a web application and learned how containers can package an application and its dependencies. This helped me understand how containers can make applications easier to move between environments. I also learned about Docker Compose and how it can be used to define and manage multiple containers that need to work together.

Another important part of this course was learning how AWS services can be used to host an application. I hosted an Angular frontend using Amazon S3 and worked with API Gateway, Lambda, and DynamoDB to build the backend functionality. I learned that deploying an application involves more than uploading files or writing code. The different services need to be configured correctly, and the frontend, API, backend functions, and database must communicate with each other.

Working with Lambda and DynamoDB also helped me understand how serverless applications operate. I created and tested Lambda functions for database operations involving questions and answers. This gave me experience connecting application functionality to cloud-based data storage and testing whether requests worked as expected. I also learned more about

API endpoints, CORS configurations, IAM roles, and permissions. These are important because an application needs to work correctly while still protecting its resources.

I believe one of my strengths as a developer is persistence. Working through this course required troubleshooting configuration problems, testing different settings, and figuring out why something was not working as expected. Cloud development can involve several services that depend on one another, so a problem in one area can affect the entire application. I learned the importance of checking each part of the system rather than assuming the problem is always in the code.

I also bring problem-solving skills and experience working through technical challenges. My Computer Science coursework has helped me build my programming and analytical skills, while my professional experience has taught me the importance of responsibility, teamwork, and following processes. I still have more to learn, especially as I gain experience with real-world applications, but I am more comfortable working with cloud services and understanding how application components fit together.

The skills I developed in CS 470 will help me pursue entry-level roles such as junior software developer, web developer, cloud support associate, or application support specialist. These positions would allow me to continue building my programming and cloud computing experience while learning from other developers. My goal is to use the knowledge I have gained through my degree and coursework to transition into a technology-focused career and continue developing my skills.

Planning for Growth

If my web application needed to support more users in the future, I would first look at how the application is currently being used and identify which parts might become bottlenecks. For example, an increase in website traffic could lead to more API requests, more Lambda invocations, and more database operations. Instead of increasing every resource immediately, I would monitor the application and use that information to decide where improvements are needed.

Microservices and serverless services could help make future growth easier to manage. By separating application responsibilities, individual components can be maintained and updated without necessarily changing the entire application. For example, the frontend could remain hosted in Amazon S3 while API Gateway routes requests to the appropriate Lambda functions. Each function could handle a specific operation, such as retrieving a question, adding an answer, or deleting a record. This separation would make it easier to troubleshoot individual functions and modify specific features as the application grows.

For scaling, I would consider how AWS services can respond to changes in demand. Lambda can scale the number of function executions as requests increase, subject to AWS service quotas and account limits. DynamoDB also offers scaling options that can support changing workloads. I would monitor request volume, errors, response times, and database usage to determine whether the application needs adjustments. If the application reached service limits or experienced performance problems, I would review those limits and adjust the configuration or architecture as needed.

Error handling would also be important as the application grows. I would make sure the frontend can display useful messages when a request fails instead of leaving users confused. The backend should validate incoming data, handle missing or invalid records, and return appropriate status codes. I would also use CloudWatch logs and metrics to help identify problems with Lambda functions and other AWS services. For failures that can be retried safely, I would consider retry settings and mechanisms for handling repeated failures. However, I would be careful with retries for operations that change data so they do not accidentally create duplicate records or repeat an action unnecessarily.

Cost prediction would be another important part of planning for growth. I would estimate the expected number of users and requests, the amount of data stored, and how frequently the application would access that data. I would then use AWS pricing information and the AWS Pricing Calculator to estimate costs before making major changes. After deployment, I would review actual usage and billing information to see whether the application was staying within its budget. Setting budgets and billing alerts would also help identify unexpected increases in spending.

Containers and serverless computing have different advantages when it comes to predictable costs. Containers can be more cost predictable when an application needs resources running continuously because the cost of the provisioned compute capacity is easier to estimate. However, containers can continue using resources even when demand is low, and the organization may need to manage scaling and infrastructure. Serverless services such as Lambda use a pay-for-use model, which can be cost-effective when requests are occasional or traffic changes significantly. On the other hand, costs can increase as the number of requests and the

amount of processing increase. Therefore, neither option is always cheaper. I would compare the expected workload, resource requirements, and pricing model before choosing an approach.

There are also advantages and disadvantages to consider when expanding an application. One advantage of serverless computing is that AWS manages much of the underlying infrastructure, allowing developers to spend more time on application features. Services can also scale with demand, which may reduce the need to maintain excess capacity. Separating the application into smaller components can make troubleshooting and updates easier.

However, serverless applications depend on the availability and configuration of cloud services, and developers must understand service limits, permissions, and possible additional costs. Containers offer more control over the application environment and can be useful for workloads that need to run continuously or require specific configurations. Their disadvantages include the need to manage container images, deployments, resource allocation, and scaling. The best choice would depend on the application's workload, complexity, budget, and maintenance needs.

Elasticity and pay-for-service would play a major role in my decisions about future growth. Elasticity allows resources to adjust as demand changes, helping an application avoid having too little capacity during busy periods or paying for excessive capacity when demand is low. Pay-for-service means costs are connected to the resources and services being used, although the exact pricing model differs by service. Together, these concepts can help balance performance and cost. I would use monitoring and usage estimates to determine when scaling or architectural changes are necessary instead of making changes based only on assumptions about future traffic.

Overall, I would plan for growth by monitoring the application, estimating costs, improving error handling, and choosing services that fit the expected workload. Cloud services provide flexibility, but that flexibility still requires planning. My goal would be to build an application that can support more users reliably without creating unnecessary expenses or security risks.